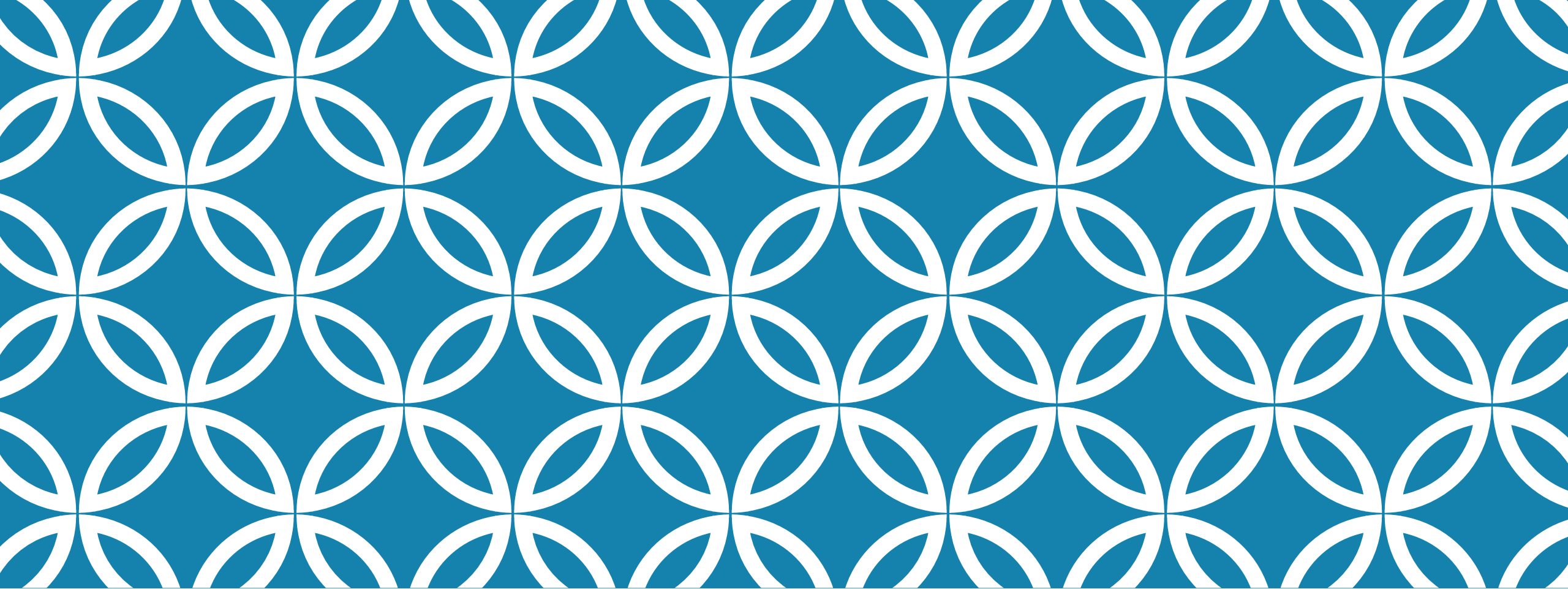




STATIC PROGRAM ANALYSIS

Lecture 18 – Working with
analyzers

SA FRAMEWORKS VS SA TOOLS

SA Frameworks

- Soot, WALA
- More flexible
- Allows you to define specific analysis to answer specific questions
- Difficult to install
- Knowledge of the framework and SA concepts

SA Tools

- Infer, CheckStyle, SonarQube, SpotBugs
- Preset set of bugs it detects
- Easy to install
- Use a black-box, unknown what type of analyses are run
- Tuning through available parameters, or annotations.

INFER

Static analyzer develop by Facebook/Meta (interprocedural + linters)

Cross-language: Java, C/C++, and Objective-C/C++

Infrastructure essences:

- Efficient modular analysis
- Clean abstraction layer
- Fluent multi-lingual frontend

Installation: <https://fbinfer.com/docs/getting-started>

- Homebrew, binaries, build your own or Docker
- To run `infer run -- javac Hello.java`
- Compilers: `javac`; `clang -c`; `gcc -c`
- Builders: `./gradlew build`; `make <make target>`; `mvn <maven target>`

INFER — MODULAR ANALYSIS

Based on the abduction technology

Instead of the main procedure, start from anywhere

When analyzing, error-free executions are gathered (e.g., arg cannot be null)

Record these requirements as a pre-condition to the method

Stitches programs parts to give the picture of the whole

INFER RUN

1. The capture phase

- Translates file into Infer's IR
- Stores intermediate files in the results directory infer-out/ in a db format

2. The analysis phase

- Files inside infer-out are analyzed
- Each method is analyzed separately
- If a potential bug is encountered in m, it stops there and continues to other methods
- Fix errors and run again

3. Reports

- Stdout and in infer-out/report.txt

SEVERAL WAYS TO INVOKE

infer

- if it has compilation command specified as “--” then it behaves as “infer run”

infer run

- = capture + analyze + report

infer capture

- translates source files into its IR for later analysis

infer analyze

- perform analysis of the captured files

infer report

- computes and manipulated results

infer-compile

- just compiles the program according to the compile command (to debug or to configure build system)

infer-debug

- prints internal data-structures
- infer debug --procedures --procedures-cfg
- <https://dreampuf.github.io/GraphvizOnline>

infer-explore

- shows symbolic program traces for errors

report

- selectively report bugs based on heuristics

ISSUES TYPE (LANGUAGE SPECIFIC -- JAVA)

Arbitrary Code Execution Under Lock

- code holds lock and executes arbitrary code (callbacks) that can acquire lock

Checkers Immutable Cast

- When an immutable collection is returned from a method whose return type is mutable (e.g., interface List vs ImmutableList)

Checkers Printf Args

- When the arguments to a printf does not match the format string.

Condition Always False/True

Several type with levels of buffer overruns

Uses several annotations:

- @PerformanceCritical
- @Expensive
- If one method is reachable from another, i.e., @PerformanceCritical does not call @Expensive

Deadlock

Divide by Zero

Redundant condition

75+ Bug Types (across all languages)

ANALYSIS TYPES

Infer contains several analyses

Each analysis focuses on finding certain bugs in the code

- Biabduction: divide by zero, null dereference
- Bufferoverflow: condition always true/false, various level of buffer overruns
- Annotation Reachability: class expensive method
- Inefficient keyset Iterator
- Loop Hoisting: Invariant call
- Pulse: interprocedural, should replace Biabduction
- Quandary (taint analysis): identifies unsafe information flows.
- RacerD: thread safety analysis

CODE EXAMPLES

<https://github.com/facebook/infer/tree/main/infer/tests/codetoanalyze/java>

Each analysis type has its code examples

Let's try some